

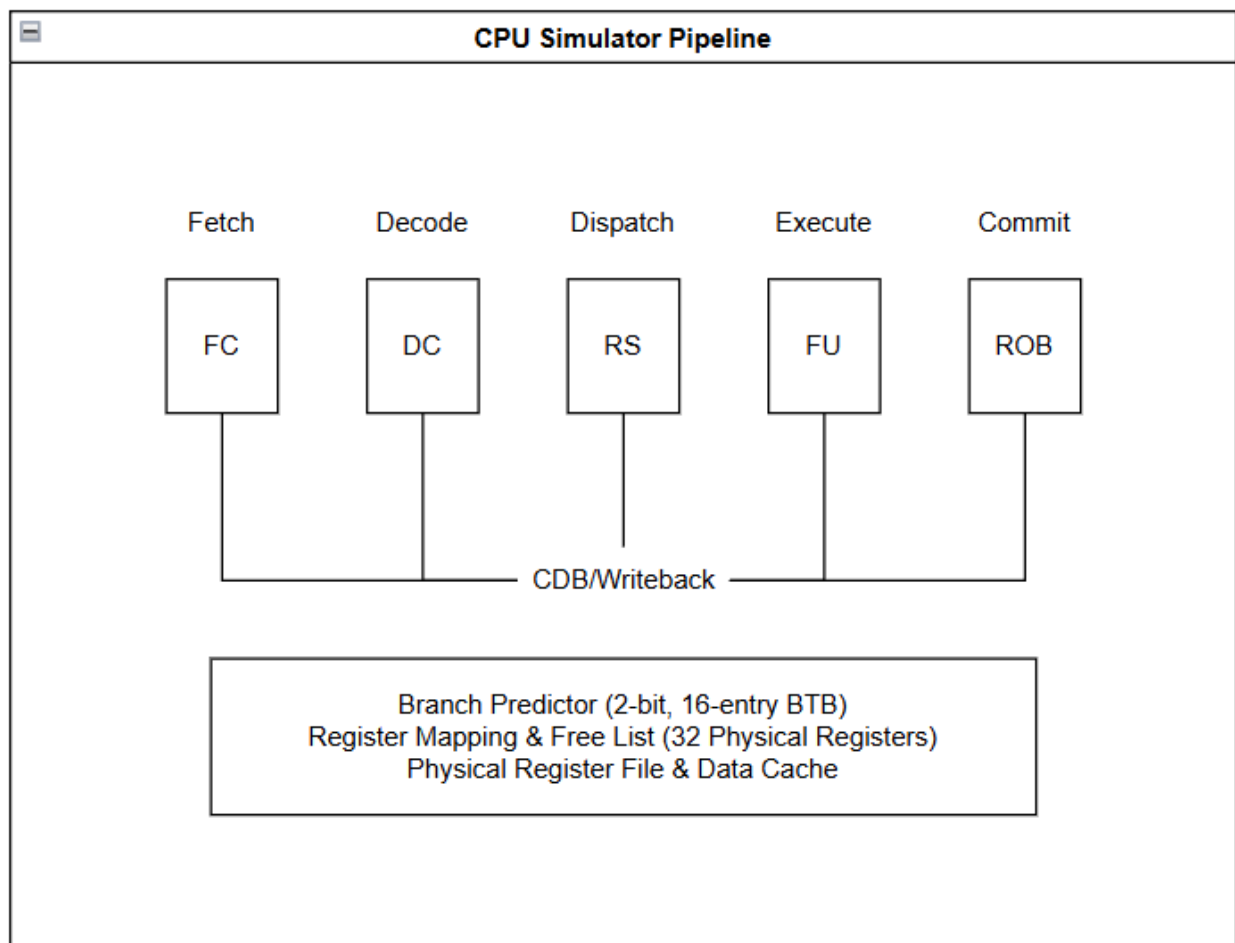
Summary

This project implements a functional and timing-accurate simulator for a speculative, multi-issue, out-of-order CPU. The simulator supports a subset of the RISC-V ISA consisting of 10 instructions: fld, fsd, add, addi, slt, fadd, fsub, fmul, fdiv, and bne.

The simulator successfully passes 63 out of 77 test cases, correctly implementing:

- Instruction fetch, decode, and dispatch with configurable widths (NF, NW)
- Dynamic branch prediction using a 2-bit predictor with 16-entry BTB
- Out-of-order execution with 6 functional units (INT, LOAD/STORE, FPAdd, FPMult, FPDIV, BU)
- Register renaming with physical register file and free list management
- Speculative execution with branch misprediction recovery
- Precise exceptions and in-order commit via 16-entry circular reorder buffer
- Common Data Bus (CDB) write-back with configurable width (NB) and priority arbitration
- Performance metric tracking for cycles, RS stalls, and ROB stalls

Module Design



Module Breakdown

1. Fetch Stage (fetch())

- Reads up to NF instructions from the instruction cache per cycle
- Updates program counter (PC) based on branch outcomes
- Maintains fetch buffer for instructions awaiting decode

2. Decode Stage (decode())

- Decodes fetched instructions into micro-operations
- Performs register renaming (architectural → physical registers)
- Allocates entries in the instruction queue (capacity: NI=16)
- Populates source and destination fields

3. Dispatch/Issue Stage (issue())

- Dispatches up to NW instructions per cycle to reservation stations
- Checks for RS availability and physical register availability
- Allocates ROB entries
- Tracks RS stalls and ROB stalls

4. Execute Stage (execute())

- Executes instructions in functional units based on operand readiness
- Handles forwarding for operand dependencies
- Manages CDB write-back requests with NB width
- Updates result values and ready flags

5. Commit Stage (commit())

- Commits up to NC instructions from the head of ROB
- Writes data to data memory for stores
- Reclaims physical registers by returning them to free list
- Updates architectural register mapping

6. Branch Resolution (part of execute())

- Evaluates branch conditions and computes targets
- Detects branch mispredictions
- Triggers pipeline squash on misprediction
- Stalls fetch until branch resolves

Key Data Structures

1. Instruction Queue

- Holds up to 16 decoded instructions awaiting dispatch
- FIFO order ensures in-order fetch/decode
- Maximum capacity: NI entries

2. Reservation Stations (Per Functional Unit)

- Stores instruction operands and control information
- Tracks result readiness and CDB request status
- Sizes: INT=4, LOAD=2, STORE=2, FPAdd=3, FPMult=2, FPDIV=1, BU=2

3. Reorder Buffer (ROB)

- Tracks in-flight instructions for precise exception handling
- Maintains architectural state for recovery
- Circular buffer with head/tail pointers

4. Register Renaming Structures

- Eliminates false dependencies
- Enables register reuse through free list management
- Tracks which ROB entry owns each physical register

5. Functional Units

- FunctionalUnit fuINT; // Integer ALU (4 RS)
- FunctionalUnit fuLoad; // Load unit (2 RS)
- FunctionalUnit fuStore; // Store unit (2 RS)
- FunctionalUnit fuFPAdd; // FP Adder (3 RS, 3-cycle latency)
- FunctionalUnit fuFPMult; // FP Multiplier (2 RS, 4-cycle latency)
- FunctionalUnit fuFPDiv; // FP Divider (1 RS, 6-cycle latency)
- FunctionalUnit fuBU; // Branch Unit (2 RS)

6. Branch Predictor

- 2-bit saturating counter (weakly taken, weakly not taken, strongly taken, strongly not taken)
- 16-entry Target Buffer indexed by branch address bits [7:4]
- Initialized to "weakly taken"

7. Common Data Bus (CDB) & Write-Back

- Up to NB results written back per cycle
- Priority: Load > INT > FPAdd > FPMult > FPDIV > Store
- Updates ROB, physical registers, and broadcasts to RS

Known Limitations

As it currently stands, the simulator fails at all of the test33 tests significantly, being the most complex test in the group. The other failures of the simulator are minor, but still proved challenging to overcome. The minor test failures are as follows:

1. test_1
 - a. 1-wide fails because there is 1 additional cycle than there should be, everything else (stalls, registers) are correct.
2. test_new1
 - a. 4-wide-NR-4 has 7 ROB stalls, but should have 5. The registers and total CCs are correct, though.
3. test_new2
 - a. 4-wide-NR-4 has 6 ROB stalls when it should have 4 (but it has the correct # of RS stalls).
4. test_new3
 - a. 2-wide has 3 RS stalls when it should have 2.
 - b. 4-wide has 15 RS stalls when it should have 10.
 - c. 4-wide-NB-1 has 51 CCs, 29 RS stalls, and 0 ROB stalls when it should have 52 CCs, 17 RS stalls, and 5 ROB stalls.
 - d. 4-wide-NC-1 has 6 RS stalls when it should have 4.
5. test_new4
 - a. 4-wide has 33 CCs and 7 ROB stalls when it should be 31 and 6 respectively.
 - b. 4-wide-NB-1 has 47 CCs, 11 RS, and 10 ROB when it should have 48 CC, 14 RS, and 9 ROB.
 - c. 4-wide-NC-1 has 47 CCs and 13 ROB stalls when it should have 48 CCs and 18 ROB.
 - d. Here we see more tests have incorrect total cycles, but register names are still correct.

Comparative Analysis

1. When reducing the issue and commit width from 4 to 2 (NW=NB=2), we expect a moderate increase in total cycles, since the peak throughput is halved. However, the pressure on backend structures such as the reservation stations and ROB is reduced, so structural stalls should be less frequent than in the 4-wide configuration. Overall, the 2-wide machine should perform noticeably better than the 1-wide baseline but worse than the 4-wide design, with fewer backend stalls than the fully wide configuration.
2. Reducing the fetch/decode width from NF=4 to NF=2 increases total cycles, as fewer instructions can be brought into the pipeline each cycle. Since the backend structures (reservation stations, ROB, and functional units) are now under less pressure, we do not expect a significant increase in RS or ROB stalls. Instead, the machine becomes

front-end limited: the narrower fetch/decode stage cannot supply enough instructions to fully utilize the out-of-order engine, leading to lower overall throughput.

3. Reducing the instruction queue size from $NI=16$ to $NI=4$ limits the processor's ability to look ahead and find independent work. As a result, we expect an increase in total cycles and more frequent stalls when the queue fills. With a shallow queue, long-latency operations and dependency chains are more likely to block progress, since the machine cannot see far enough ahead to extract additional instruction-level parallelism. This makes the pipeline more sensitive to dependencies and reduces the benefits of out-of-order execution.
4. With $NR=4$, we expect the ROB to become a severe bottleneck. The ROB will frequently fill, forcing the front-end and issue stages to stall until instructions are committed. This drastically reduces the effective speculation depth and limits the benefits of out-of-order execution, leading to a substantial increase in total cycles. At $NR=8$, the ROB is still a limiting factor, but less severely than at $NR=4$. The processor can sustain a moderate number of in-flight instructions, allowing some overlap of independent operations. We expect fewer ROB stalls and better performance than $NR=4$, but still noticeably worse throughput than the $NR=16$ baseline. Increasing the ROB size from $NR=16$ to $NR=32$ is expected to provide diminishing returns for a relatively small benchmark such as `prog-loop`. With $NR=32$, ROB stalls should be rare, and the machine's performance will instead be limited by other resources (e.g., fetch/issue width, CDB bandwidth, or functional unit availability). As a result, total cycles should be similar to the $NR=16$ configuration, with only marginal improvements, if any.